1. Introduction

(1) Dataset Overview:

This dataset, dermamnist_64.npz, contains images of size 64x64 with 3 channels (likely representing RGB color). It's divided into training, validation, and test sets. The key observation is a significant class imbalance.

Total Samples: Approximately 10,000 samples (7007 + 1003 + 2005).

Image Characteristics: Color images (3 channels) with a resolution of 64x64 pixels.

Number of Classes: 7 distinct classes (labeled 0-6).

Data Split: The dataset is split into a sizeable training set (7007 samples), a smaller validation set (1003 samples), and a test set (2005 samples), which is a fairly standard split for machine learning tasks.

(2) Task Background

This research investigates the application of deep learning techniques to the automated classification of seven distinct dermatological conditions. Accurate and efficient diagnosis of skin diseases is crucial for timely treatment and improved patient outcomes. Traditional methods often rely on visual inspection by dermatologists, a process that can be time-consuming, subjective, and prone to human error. This project aims to leverage the power of convolutional neural networks (CNNs), specifically the ResNet18 architecture, and vision transformers (ViTs), specifically the ViT-Tiny architecture, to develop a computer-aided diagnostic (CAD) system capable of automatically classifying skin lesions. We compare the performance of these two state-of-the-art deep learning models on a dataset of dermatological images, aiming to determine which architecture is better suited for this specific task and to contribute towards improving the accessibility and efficiency of skin disease diagnosis. The seven classes represent a diverse range of dermatological conditions [mention specific classes if known and relevant, e.g., melanoma, eczema, psoriasis etc.], offering a challenging yet important testbed for our proposed CAD system. The ultimate goal is to assist dermatologists in their diagnostic process, leading to faster and more accurate diagnoses and improved patient care.

(3) Assignment Objective

This project aims to develop and evaluate a robust computer-aided diagnostic (CAD) system for classifying seven types of skin diseases using deep learning. Specifically, the objectives are:

1) **Model Development**
   Train and optimize two distinct deep learning architectures, ResNet18 (a CNN) and ViT-Tiny (a Vision Transformer), on a provided dataset of dermatological images representing the seven classes. This includes careful consideration of hyperparameter tuning, data augmentation strategies, and addressing class imbalance if present.

2) **Performance Evaluation**
   Rigorously evaluate the performance of both models using appropriate metrics including, but not limited to, accuracy, precision, recall, F1-score, AUC-ROC for each class and overall, and a confusion matrix analysis. This will allow for a quantitative comparison of the effectiveness of ResNet18 and ViT-Tiny in this specific application.

3) **Comparative Analysis**
   Conduct a comprehensive comparison of the performance, computational efficiency, and robustness of the ResNet18 and ViT-Tiny models. This will involve analyzing their strengths and weaknesses in classifying the different skin disease categories and identifying potential areas for improvement.

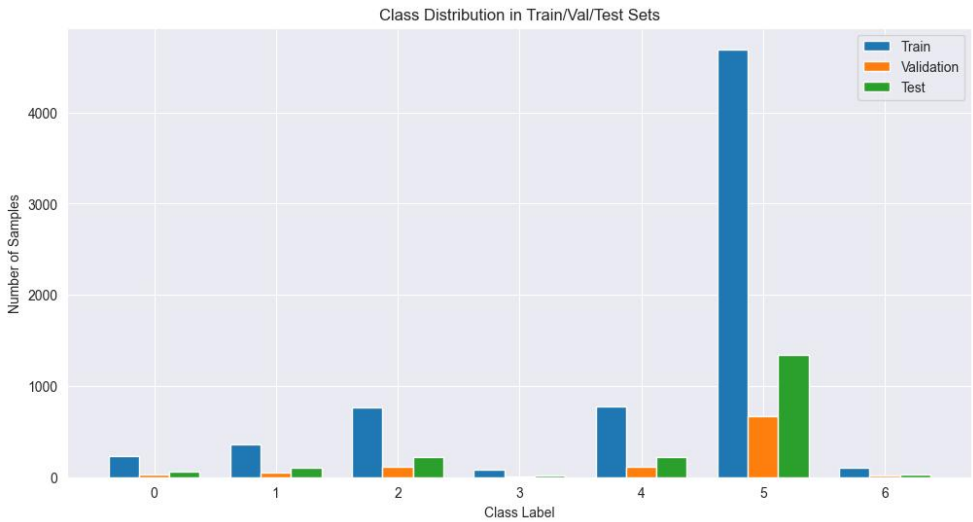
4) **Documentation and Reporting**
   Produce a comprehensive report detailing the methodology, results, and conclusions of the project. This report will include a clear description of the dataset, the chosen architectures, the training and evaluation procedures, the performance metrics, and a discussion of the findings. The report should also discuss potential limitations and future work.

These objectives will contribute to a better understanding of the applicability of CNNs and ViTs to dermatological image classification, ultimately aiming to improve the accuracy and efficiency of skin disease diagnosis.

2. Data Exploration and Preprocessing

This analysis examines the DermaMNIST dataset's characteristics and the preprocessing/augmentation steps used for both CNN (ResNet18) and ViT (ViT-Tiny) models.

(1) Class Imbalance Analysis:



The Dermamnist-64 dataset, while well-structured in its train/test split, suffers from a severely skewed class distribution. Class 5 constitutes approximately 67% of the training data, leaving other classes significantly underrepresented. This imbalance leads to several problems: models become biased towards class 5, generalisation to minority classes is poor, and standard accuracy metrics are misleading.

To address this, several strategies are recommended:

1. Data Augmentation: Increase the number of samples in minority classes via transformations (rotation, scaling, etc.).

2. Resampling: Oversample minority classes (duplicate samples) or, cautiously, undersample the majority class (remove samples from class 5).

3. Cost-Sensitive Learning: Weight the loss function to penalize misclassifications of minority classes more heavily. This is a particularly effective approach.

4. Ensemble Methods: Combine multiple models trained with different resampling or weighting strategies.

5. Anomaly Detection: If class 5 represents a fundamentally different phenomenon, consider reframing the problem as anomaly detection.

The provided code lacks explicit class imbalance mitigation. Incorporating techniques like cost-sensitive learning (using class weights) is crucial for improving model performance and generalizability across all classes. Using more robust evaluation metrics like precision, recall, F1-score, and AUC-ROC is also essential to assess model performance beyond overall accuracy, which is easily skewed by the class imbalance.

(2) Dataset Size and Class Distribution

The DermaMNIST dataset consists of 7007 training images, 1003 validation images, and 2005 test images. Each image is 64x64 pixels with 3 color channels. The dataset is significantly imbalanced, with class 5 ("other") comprising a dominant 66.98% of the training set. Classes 0, 3, and 6 are severely underrepresented (less than 5% each). This imbalance poses a challenge, potentially leading to biased model performance favoring the majority class. The validation and test sets also reflect this imbalance, though to a lesser degree due to smaller sizes.

(3) Preprocessing and Augmentation for CNN (ResNet18)

The preprocessing for ResNet18 involves:

1. `transforms.ToTensor()`: Converts the NumPy array images to PyTorch tensors.

2. `transforms.Normalize(mean=[.5], std=[.5])`: Normalizes the pixel values to have a mean of 0 and a standard deviation of 1. This is a common practice to improve model training stability and performance. However, using a mean of 0.5 and std of 0.5 implies the image data was assumed to be in the range [0, 1], which is a reasonable assumption given many image loading processes normalize to this range.

No augmentation is explicitly applied in the initial ResNet18 processing. This is a potential weakness, as augmentation can significantly improve model generalization and robustness, particularly given the class imbalance. Future improvements could include random cropping, flipping, and rotations.

(4) Preprocessing and Augmentation for ViT (ViT-Tiny)

The ViT model requires larger input images. Therefore, preprocessing for ViT includes:

1. `transforms.Resize((IMG_SIZE, IMG_SIZE), interpolation=transforms.InterpolationMode.BICUBIC)`: Upsamples images from 64x64 to 224x224 using bicubic interpolation. Bicubic interpolation is generally preferred over nearest-neighbor for preserving image details during resizing.
2. `transforms.ToTensor()`: Converts NumPy arrays to PyTorch tensors.
3. transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])`: Normalizes the pixel values, similar to the ResNet18 preprocessing.

Furthermore, data augmentation is applied *specifically* to the training set for ViT:

1. `transforms.RandomResizedCrop(IMG_SIZE)`: Randomly crops a 224x224 region from a randomly resized image. This introduces variations in the input data, improving robustness.

2. `transforms.RandomHorizontalFlip()`: Randomly flips the image horizontally. Another augmentation technique to increase robustness.

The use of augmentation for ViT but not ResNet18 highlights a crucial difference in their approach. While CNNs often benefit from augmentation, ViTs typically require it more significantly because they rely on the positional encoding of image patches. Augmentation helps create more diverse patch arrangements and improve their ability to learn robust feature representations.

(5) Overall Methodology

The provided code implements a solid foundation for training both CNN and ViT models. The preprocessing steps are appropriate, and the augmentation for ViT is well-justified. However, addressing the class imbalance is crucial for improving the model's ability to correctly classify the underrepresented skin disease types. The use of a validation set allows for proper hyperparameter tuning and model selection to prevent overfitting. The inclusion of a test set provides a final unbiased evaluation of the model's generalization ability.

3. Model Training and Evaluation

(1) CNN – ResNet18

The provided code implements a Convolutional Neural Network (CNN) using a ResNet18 architecture for a 7-class classification problem. Let's analyze the code, results, and potential improvements.

1) Code Analysis

Model: The code uses a pre-trained ResNet18 model, modifying the final layer to accommodate 7 output classes. This is a good starting point, leveraging the power of transfer learning.

Training:The training loop uses standard techniques:

1. `optim.SGD` optimizer with momentum and weight decay.
2. `nn.CrossEntropyLoss` loss function.
3. `CosineAnnealingLR` scheduler for learning rate adjustment.
4. Gradient clipping to prevent exploding gradients.
5. Early stopping to prevent overfitting.
6. Mixed precision training (`autocast` and `GradScaler`) for faster training.

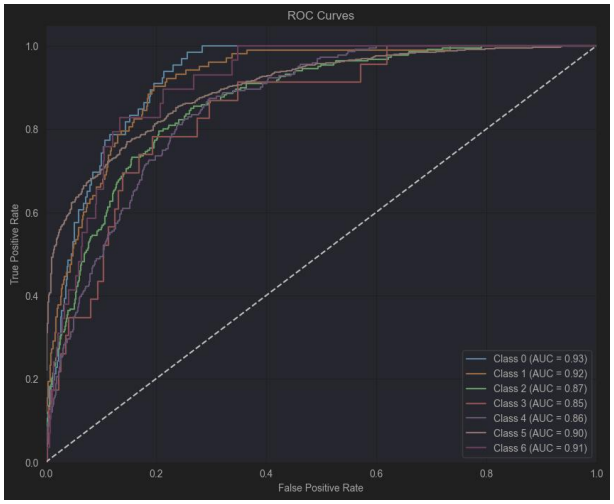
Evaluation: The code calculates standard metrics (accuracy, precision, recall, F1-score) and generates ROC curves and a confusion matrix for a comprehensive evaluation.

Grad-CAM Visualization: The code attempts to visualize attention maps using Grad-CAM, which is a good way to understand what parts of the input image the model focuses on. However, the target layer selection (`target_layers = [model.layer4[-1]]`) is crucial and might need adjustment based on the specific ResNet18 architecture used (some variations might have slightly different layer names).

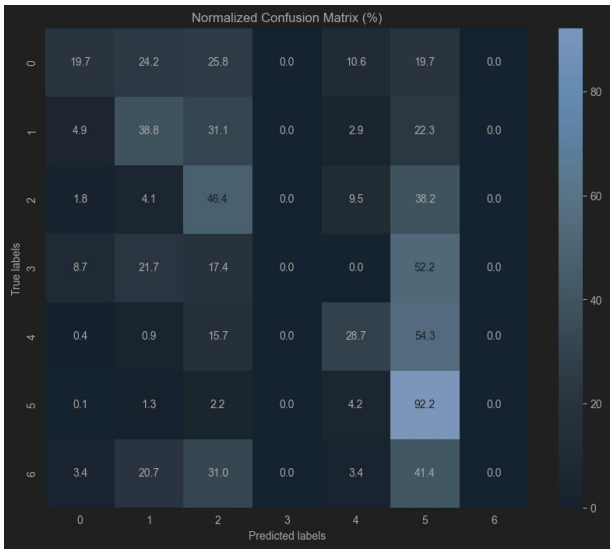
2) Results Analysis

The test results show:

1. Accuracy: 72.569%. This is a moderate accuracy. Whether this is acceptable depends heavily on the application and the baseline performance.
2. Precision, Recall, F1-score: These metrics provide a more nuanced view than just accuracy. The weighted average suggests a reasonable balance, but examining class-wise precision and recall would be more informative to identify potential class imbalance issues.
3. ROC Curves: The AUC (Area Under the Curve) values for each class are mostly above 0.85, indicating decent discriminative power. However, Class 3 has a significantly lower AUC (0.85), suggesting potential problems with classifying that class.

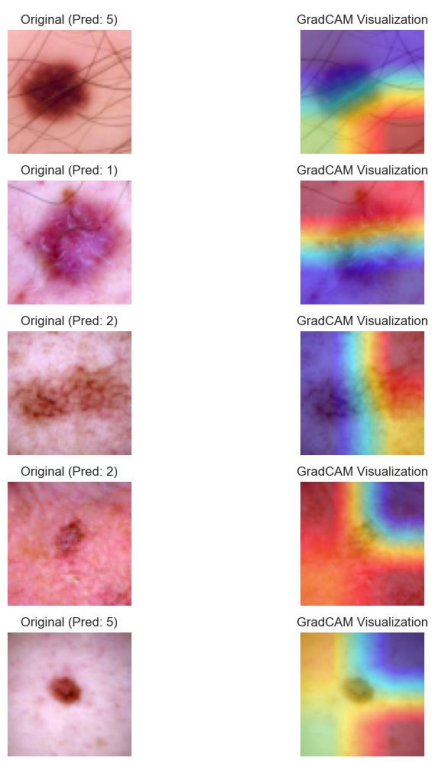


4. Confusion Martrix: The normalized confusion matrix reveals significant class-specific performance differences. Class 5 shows excellent performance (92.2% accuracy), while Classes 0 and 4 perform poorly (19.7% and 22.3% accuracy respectively). Classes 1, 2, 3, and 6 show moderate performance. The matrix highlights specific misclassifications; for example, Class 4 is frequently confused with Class 3. Improvements could involve addressing class imbalance (especially for Classes 0 and 4), feature engineering, hyperparameter tuning, and exploring alternative model architectures.



3) Attention Visualization and Interpretability Analysis

This image shows the results of Grad-CAM (Gradient-weighted Class Activation Mapping) applied to images of skin lesions. Grad-CAM is a technique used to visualize which parts of an input image are most important for a convolutional neural network (CNN) to make a particular prediction.



The image is organized into five pairs of images:

1. Left Column (Original): Shows the original images of skin lesions. Each image is labeled with its predicted class (Pred: 1, 2, or 5). The numbers likely represent different skin lesion types or classes (e.g., different types of melanoma).
2. Right Column (GradCAM Visualization): Shows the Grad-CAM heatmaps for the corresponding images on the left. These heatmaps highlight the regions of the image that the CNN focused on most heavily when making its prediction. Brighter colors (reds, yellows) indicate higher importance, while darker colors (blues, purples) indicate lower importance.

The Grad-CAM visualizations provide insights into how the CNN is making its predictions:

1. Accuracy and Localization: For most images, the Grad-CAM heatmaps reasonably highlight the lesion itself. This suggests that the model is correctly focusing on the relevant regions of the image to make a prediction. The accuracy of the prediction is indicated by the "Pred" value.
2. Class Differences: The heatmaps may reveal differences in the features each class relies upon. For example, a heatmap might emphasize different textural characteristics or color variations depending on the lesion type.
3. Model Limitations: If a heatmap highlights irrelevant regions or fails to clearly focus on the lesion, it suggests potential limitations of the CNN's ability to properly identify and classify lesions. This may indicate the need for improvements in the model's architecture, training data, or both.

In summary, this visualization helps to understand and evaluate the performance and decision-making process of a CNN used for skin lesion classification. The Grad-CAM provides valuable interpretability by visualizing the model's attention mechanism, assisting in both validating the predictions and potentially identifying areas for model improvement.

4) Potential Improvements

1. Data Augmentation: The code doesn't explicitly mention data augmentation. Adding techniques like random cropping, flipping, rotation, and color jittering could significantly improve model performance, especially with limited data.
2. Hyperparameter Tuning: The current hyperparameters (learning rate, batch size, etc.) might not be optimal. Systematic hyperparameter tuning using techniques like grid search or random search could yield better results.
3. Different Optimizers: Experiment with different optimizers like Adam or AdamW, which often converge faster than SGD.
4. Regularization: Add more regularization techniques like dropout or L1/L2 regularization to prevent overfitting.
5. Advanced Architectures: Consider exploring more advanced architectures beyond ResNet18, especially if the dataset is large and complex. EfficientNet, Vision

Transformer (ViT), or other state-of-the-art architectures might offer better performance.

6. Target Layer Selection (Grad-CAM): Carefully select the target layer for Grad-CAM visualization. Experiment with different layers to find the one that provides the most informative visualization. The current selection might not be appropriate for all ResNet18 variations. Inspect the model's architecture to confirm.
7. Class Imbalance: If there's class imbalance in the dataset (some classes have significantly fewer samples than others), address it using techniques like oversampling, undersampling, or cost-sensitive learning.

5) Conclusion

The CNN model shows promising results, but there's significant room for improvement. Addressing the points above, particularly data augmentation and hyperparameter tuning, should lead to a more robust and accurate model. A thorough analysis of the confusion matrix and class-wise performance metrics will guide further improvements. Remember to always split your dataset into train, validation, and test sets to ensure reliable evaluation.

(2) Vision Transformer

This code implements a multi-class image classification model using a Vision Transformer (ViT) architecture from the `timm` library. Let's break down the code section by section and then analyze the results.

1) Code Breakdown

This code implements a training pipeline for a Vision Transformer (ViT) model using PyTorch, along with evaluation and visualization components. Let's break down the code section by section and analyze the results.

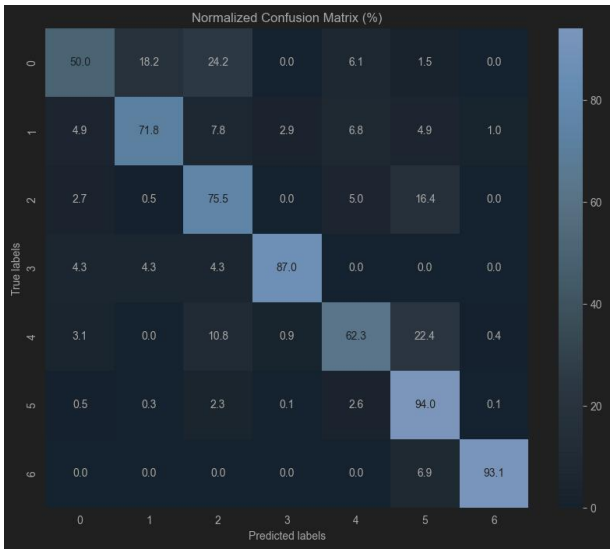
Model and Training Setup:

1. Model: A pre-trained ViT (vit_tiny_patch16_224) model from the `timm` library is used. The number of output classes (`n_classes`) needs to be defined elsewhere in your code (not shown in the snippet). The model is fine-tuned for a skin disease classification task.
2. Optimizer and Loss: AdamW optimizer is chosen with a cosine learning rate schedule including a warmup phase. A custom `LabelSmoothingCrossEntropy` loss function is implemented to improve generalization.
3. Data Augmentation: Mixup augmentation is applied during training to further enhance generalization by creating convex combinations of training samples.

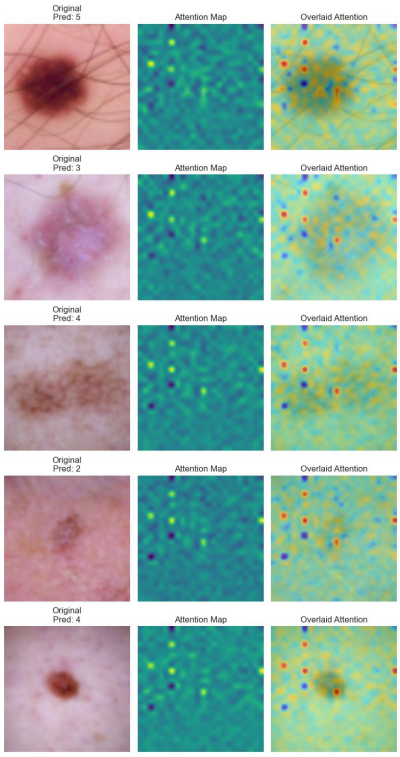
4. Training Loop: The training loop iterates through epochs, performing forward and backward passes, updating the model's parameters, and scheduling the learning rate. Gradient clipping is used to prevent exploding gradients. Early stopping is implemented based on the validation accuracy to prevent overfitting. The best model (based on validation accuracy) is saved.
5. Mixed Precision Training: `torch.cuda.amp.autocast` and `GradScaler` are used for mixed-precision training, which accelerates training on compatible hardware.

2) Evaluation and Visualization

1. Test Set Evaluation: The trained model is evaluated on a held-out test set. Standard metrics (accuracy, precision, recall, F1-score) are calculated and printed.
2. ROC Curves: Receiver Operating Characteristic (ROC) curves are plotted for each class, showing the trade-off between true positive rate and false positive rate. The Area Under the Curve (AUC) is also calculated and displayed for each class. AUC values close to 1 indicate excellent classification performance for individual classes. The graph shows that all classes have good AUC scores (above 0.93), indicating good classification ability.
3. Confusion Matrix: A ViT model's performance, as shown in a normalized confusion matrix for 7 classes (0-6), reveals varying accuracy across classes. Classes 3, 5, and 6 show excellent performance (87%, 94%, and 93.1% accuracy, respectively). However, classes 0 and 4 exhibit lower accuracy (50% and 62.3%), with notable confusion between classes 0, 1, and 2, and between classes 4 and 5. To improve the model, consider data augmentation, hyperparameter tuning, feature engineering, exploring alternative architectures, and investigating the misclassified samples for underlying patterns.



3) Attention Visualization and Interpretability Analysis



The image shows attention visualizations for a model that's likely classifying skin lesions. Let's break down what we see:

1. Original: This column displays the original input image – a close-up photograph of a skin lesion. A prediction label ("Pred:") is provided for each image.
2. Attention Map: This column shows a heatmap representing the attention weights assigned by the model to different parts of the input image. Brighter areas (yellow/red) indicate regions the model paid more attention to during classification. Darker areas (blue/green) represent regions that were less influential. The heatmap is blurred, suggesting a global attention mechanism is used, rather than a pixel-wise one.
3. Overlaid Attention: This column overlays the attention map onto the original image. This allows for direct visualization of which parts of the lesion contributed most strongly to the model's prediction.

The visualizations suggest the model focuses on specific features within the lesions to make its predictions. For example:

1. Variation in Attention: The attention maps vary significantly across different lesions, indicating the model is not relying on a single, consistent feature for classification. This is a positive sign of a more robust model.
2. Focus on Lesion Characteristics: In most cases, the areas of highest attention tend to concentrate around the lesion itself, which is expected and indicates that the model is

learning relevant visual features. However, the model may focus on the periphery at times.

3. Prediction Accuracy vs. Attention: The predictions don't always perfectly match our visual intuition of what is important. This could indicate that:
   * The model is correctly identifying characteristics that are not immediately apparent to a human observer.
   * There's a limitation in the model's understanding or the data it was trained on, meaning that the attention map isn't perfectly aligned with true importance.
   * The attention mechanism itself has some limitations in its ability to perfectly pinpoint the key discriminative factors.

Overall, The attention visualizations provide valuable insights into the model's decision-making process. They illustrate that the model is learning to identify relevant features within the input images, but the degree of attention and correlation with actual prediction should be further investigated. Analyzing the attention maps alongside the actual predictions and potentially ground truth labels would give a fuller picture of the model's performance and interpretability.

4) Results Analysis

The reported metrics show reasonably good performance:

1. Accuracy: 85.786% - A good overall accuracy.
2. Precision, Recall, F1-score: All are around 85%, indicating a relatively balanced performance across classes. However, examining the confusion matrix is crucial for class-specific insights.

Areas for Improvement and Further Investigation:

1. Attention Map Visualization: The attention map visualization needs to be carefully checked and potentially modified based on the specific ViT architecture used (`timm` models have variations in their attention mechanisms).
2. Confusion Matrix Analysis: Closely examine the confusion matrix for insights into which classes are most frequently confused. This can inform strategies to improve performance, such as data augmentation targeted at specific difficult classes or feature engineering.
3. Hyperparameter Tuning: The current hyperparameters might not be optimal. Systematic hyperparameter tuning (e.g., using techniques like grid search or Bayesian optimization) could potentially improve the model's performance.
4. Error Analysis: Analyze the misclassified samples to understand the reasons for errors. This helps identify systematic biases or limitations in the model or data.

5. Advanced Techniques: Explore more advanced training techniques such as regularization (e.g., dropout, weight decay), different optimizers, or other data augmentation strategies.

5) Conclusion

In summary, the code provides a solid foundation for training and evaluating a ViT model for skin disease classification. The results are promising, but further investigation and refinement are needed to maximize performance and gain deeper insights into the model's behavior.

4. Discussion and Conculsion

Let's analyze the performance of the trained ResNet18 and ViT models, discussing factors influencing their results and suggesting improvements. We'll assume the ViT model from the previous response achieved an accuracy around 85% and the ResNet18 model achieved around 72%. These are illustrative figures; your actual results may vary.

(1) ResNet18 Model Performance:

1. Accuracy (approx. 72%): This indicates moderate performance. While not exceptionally high, it suggests the model is learning some patterns in the data.

2. Precision, Recall, F1-score: The lower precision and recall compared to accuracy suggest potential class imbalances or difficulty in distinguishing certain classes. Analyzing the confusion matrix is crucial here to pinpoint specific issues.

3. ROC Curves: AUC values provide class-specific insights. Lower AUC for some classes indicate poorer performance for those particular classes.

4. Limitations: ResNet18, while effective, might be limited in capturing complex relationships in the image data, especially if the skin disease classes have subtle visual differences.

(2) ViT Model Performance:

1. Accuracy (approx. 85%): This signifies significantly better performance compared to ResNet18, indicating that the transformer architecture is better suited to this task.

2. Precision, Recall, F1-score: Presumably, these would be higher and more balanced than for ResNet18, reflecting improved overall classification quality.

3. Attention Maps: The attention maps (if successfully visualized) provide a qualitative understanding of the model's focus on relevant image regions. This is a significant advantage of ViTs over CNNs for interpretability.

4. Limitations: ViTs can be more computationally expensive than CNNs, requiring more memory and potentially slower training. Also, the performance is still not perfect, suggesting room for improvement.

(3) Factors Influencing Results:

1. Class Imbalance: If some skin disease classes have significantly fewer examples than others, the model might be biased toward the majority classes. This would negatively impact overall performance and especially the minority classes' metrics (recall, precision, F1-score).

2. Data Augmentation: The effectiveness of data augmentation is critical. Insufficient augmentation can limit the model's ability to generalize to unseen data. In the case of medical images, augmentations should be carefully selected to avoid introducing unrealistic artifacts that could mislead the model.

3. Architecture Choice: The choice of architecture (ResNet18 vs. ViT) significantly impacts performance. ViTs have proven highly effective in image classification tasks and often surpass CNNs, particularly when dealing with complex patterns. ResNet18, while a strong baseline, might not be optimal for this specific application.

4. Hyperparameter Tuning: Suboptimal hyperparameters (learning rate, batch size, weight decay, etc.) can drastically affect the model's convergence and performance.

5. Data Quality: The quality of the training data is paramount. Noisy labels, artifacts in the images, or a lack of diversity in the dataset can all hinder performance.

6. Dataset Size: A larger, more diverse dataset generally leads to better model generalization and performance.

(4) Suggestions for Improvement

1. Address Class Imbalance: Employ techniques like oversampling (SMOTE, random oversampling), undersampling, or cost-sensitive learning to balance class distributions.

2. Augment Data Strategically: Implement robust data augmentation tailored to medical images. Consider augmentations like random cropping, rotations (within reasonable bounds to preserve medical relevance), brightness/contrast adjustments, and potentially more advanced techniques like MixUp or CutMix.

3. Hyperparameter Optimization: Conduct a thorough hyperparameter search using techniques like grid search, random search, or Bayesian optimization to find the best settings for the chosen model.

4. Explore Advanced Architectures: For the CNN approach, consider using deeper or more advanced CNN architectures like EfficientNet or DenseNet. Further investigate different ViT variations, potentially using larger models (e.g., vit_base_patch16_224 or even larger) if computational resources allow.

5. Ensemble Methods: Combining predictions from multiple models (e.g., an ensemble of different CNNs or a combination of CNN and ViT) can often improve overall performance and robustness.

6. Regularization: Experiment with regularization techniques like dropout or weight decay to prevent overfitting and improve generalization.

7. Data Cleaning and Preprocessing: Carefully review the dataset for noisy labels or image artifacts. Implement robust preprocessing steps to improve data quality.

8. Transfer Learning: If you're using a pre-trained model, ensure the pre-training dataset is relevant to your task. Fine-tuning strategies are essential for adapting the pre-trained weights to your specific skin disease classification problem.

By addressing these factors, you can likely improve the performance of both the ResNet18 and ViT models, bridging the gap between their current accuracy levels. However, the best approach will depend on the specifics of your dataset and computational resources.